



AOP, Spring AOP



Проверить, идет ли запись

Меня хорошо видно && слышно?



Ставим "+", если все хорошо
"-", если есть проблемы

Тема вебинара

AOP, Spring AOP



Владимир Иванов

Senior Java Developer

Об опыте:

20 лет опыта java разработчиком, из них 11 на Spring Framework

Эксперт депозитарным системам в компании СМА

Telegram: @ivanovvl

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в Telegram



Задаем вопрос
в чат или ГОЛОСОМ



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом



Документ



Ответьте себе или
задайте вопрос

Цели вебинара

К концу занятия

1.	Выясним, что такое аспекты
2.	Разберемся как работают аспекты в Spring
3.	Узнаем, что нужно и что не нужно писать на аспектах)

Чуть-чуть комментариев

- Срок проверки ДЗ – не более трёх рабочих дней. Но иногда бывают завалы)
- Если делаете задание/исправление – отпишитесь об этом в чате ДЗ
- Каждое сообщение в чат принятого ДЗ переводит его в статус “На проверку”
- Если что не так – есть чат группы)
- Или ЛС комьюнити-менеджера, а так же мой

Репозиторий с примерами

Давайте скачаем репозиторий и откроем проект

1. `git clone git@github.com:OtusTeam/Spring.git`
`cd ./<имя группы>/spring-04-aop`
2. Открываем `spring-04-aop/aop-classwork/`

План темы AOP:

- Шутка (Cross-cutting)
- АОП
- Аспект
- AspectJ, Spring AOP
- Прокси

AOP

Пара слов про тему AOP:

- Это будете использовать напрямую достаточно редко
- Но это базовая магия, которой нужно владеть
- Весь Spring основан на AOP
- Это такая же базовая функциональность как IoC-контейнер

Маршрут вебинара

Cross-cutting

Aspects, Proxy

Weaving

Состав аспекта

Advices

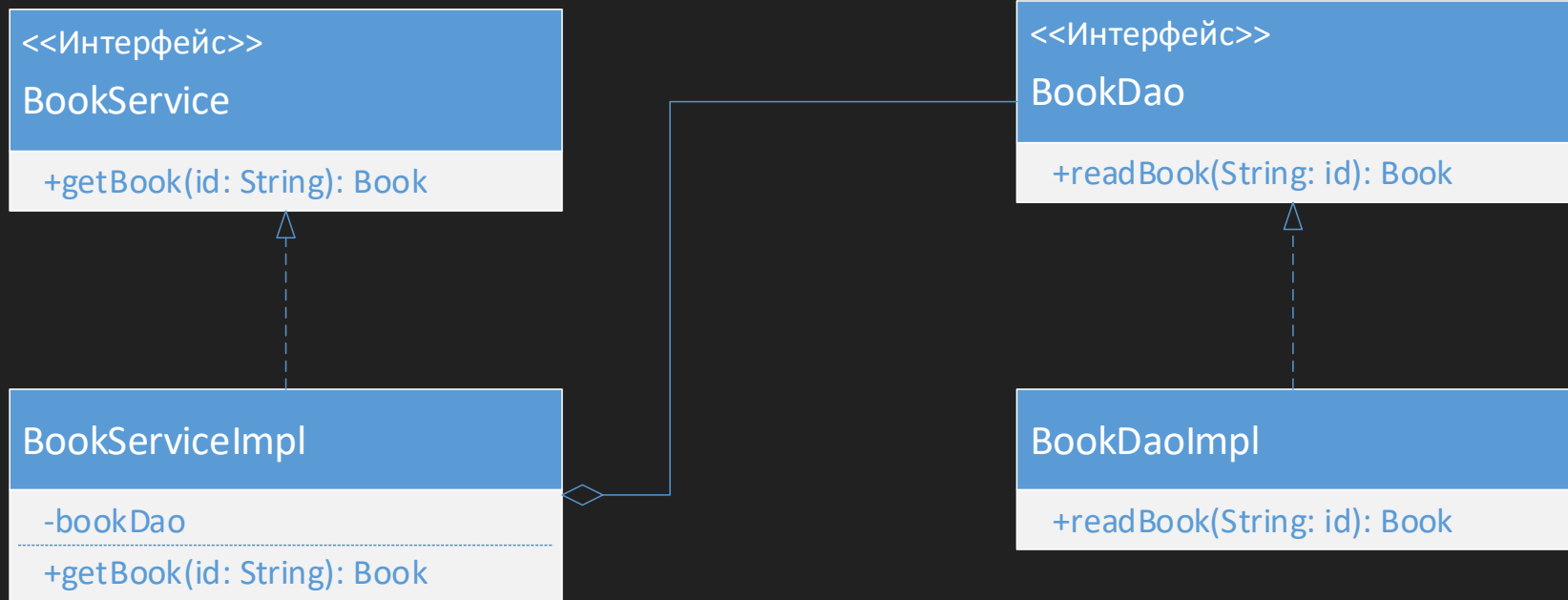
AspectJ pointcuts

Нативные аспекты

Best, Bad и Быт Practices

Cross-cutting функциональность

Диаграмма классов шутки



Шутка смешная (нет) <https://habr.com/post/114649/>

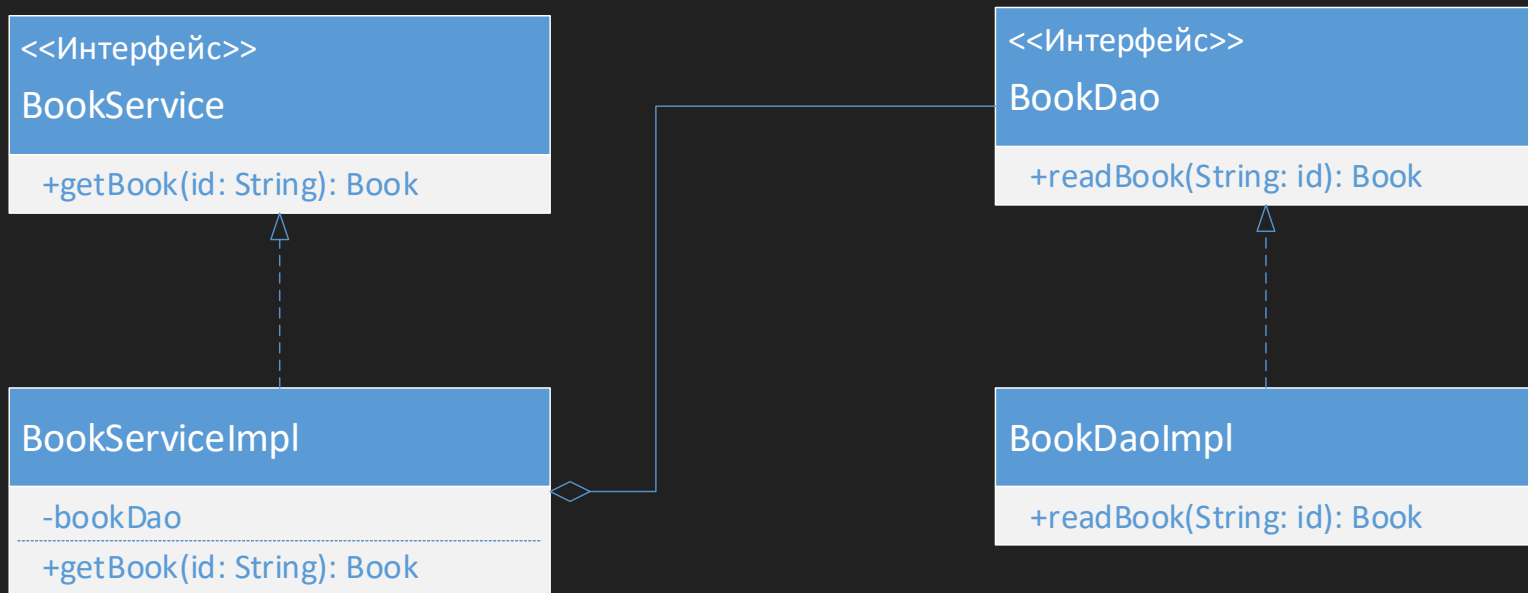
```
@Service
public class BookServiceImpl implements BookService {

    private final BookDao bookDao;

    public BookService(BookDao bookDao) {
        this.bookDao = bookDao;
    }

    @Override
    public Book getBook(String bookId) {
        Book book = bookDao.readBook(bookId);
        return book;
    }
}
```

Диаграмма классов шутки



Будем дальше писать
BookServiceImpl.getBook

Шутка

```
@Override  
public Book getBook(String bookId) {  
    Book book = bookDao.readBook(bookId);  
    return book;  
}
```

Шутка

```
@Override
public Book getBook(String bookId) {
    LOG.debug("Call method getBook with id={}", bookId);

    Book book = bookDao.readBook(bookId);

    LOG.debug("Book is {}", book);
    return book;
}
```


Шутка

```
@Override
public Book getBook(String bookId) {
    LOG.debug("Call method getBook with id={}", bookId);

    Book book = null;
    try {
        book = bookDao.readBook(bookId);
    } catch (SQLException e) {
        throw new ServiceException(e);
    }

    LOG.debug("Book is {}", book);
    return book;
}
```



Шутка

```
@Override
public Book getBook(String bookId) {
    if (!SecurityContext.getUser().hasRight("GetBook"))
        throw new AuthException("Access Denied");

    LOG.debug("Call method getBook with id={}", bookId);

    Book book = null;
    try {
        book = bookDao.readBook(bookId);
    } catch (SQLException e) {
        throw new ServiceException(e);
    }

    LOG.debug("Book is {}", book);
    return book;
}
```



Шутка

```
@Override
public Book getBook(String bookId) {
    if (!SecurityContext.getUser().hasRight("GetBook"))
        throw new AuthException("Access Denied");

    LOG.debug("Call method getBook with id={}", bookId);

    String cacheKey = "getBook:" + bookId;
    Book book = null;
    try {
        if (cache.containsKey(cacheKey)) {
            book = (Book) cache.get(cacheKey);
        } else {
            book = bookDao.readBook(bookId);
            cache.put(cacheKey, book);
        }
    } catch (SQLException e) {
        throw new ServiceException(e);
    }

    LOG.debug("Book is {}", book);
    return book;
}
```

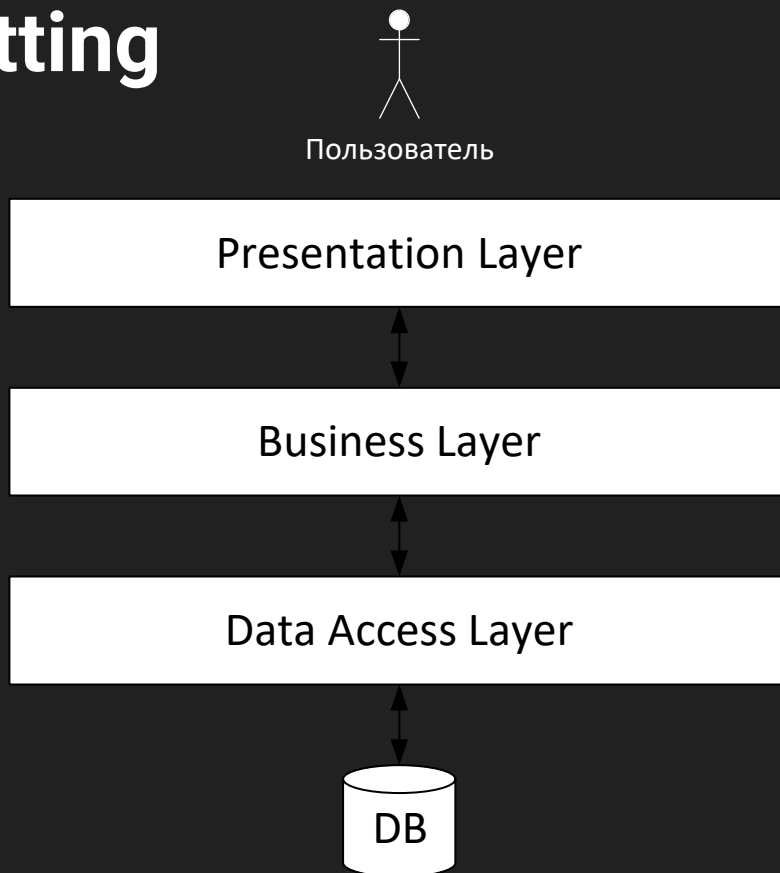
Шутка

- Сколько строчек бизнес логики?
- Сколько строчек вспомогательной логики?
- Море сервисного кода
- И это только начало
- И это только один метод
- В только одном сервисе
- В одном архитектурном слое

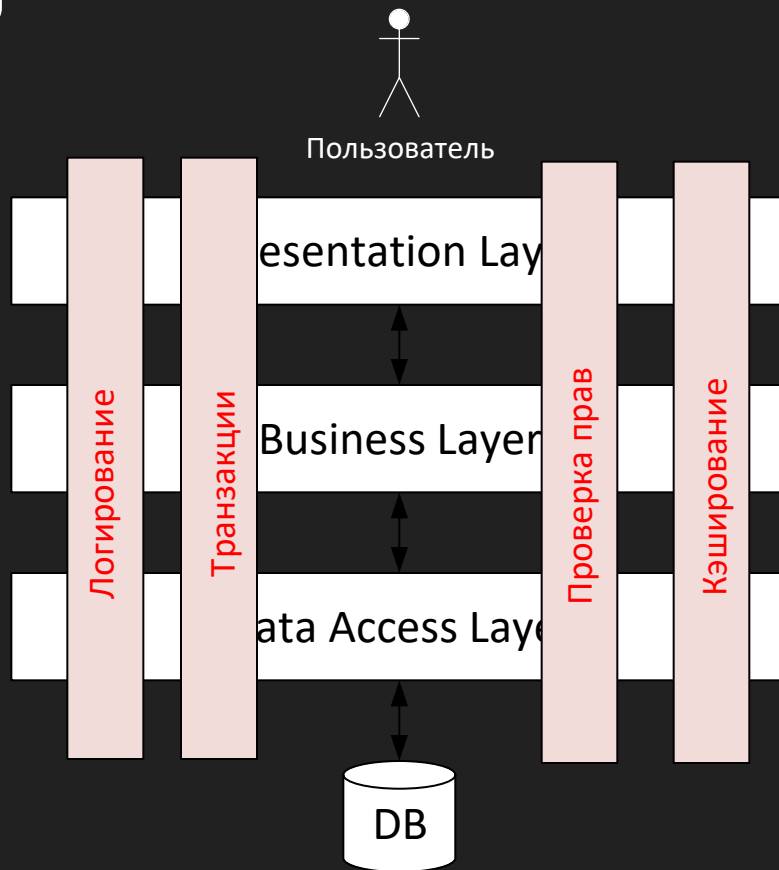
Cross-cutting

- Такая функциональность называется ортогональной или сквозной (cross-cutting)
- Логгирование – отличный пример cross-cutting функциональности

Cross-cutting



Cross-cutting



Cross-cutting

- Логгирование
- Авторизация
- Exception Handling
- Транзакции
- Кэширование
- Benchmarking

БИЗНЕС-ЛОГИКИ ЗДЕСЬ НЕТ !!!

АОП

- В ООП есть решение проблемы cross-cutting функциональности, но его не всегда удобно использовать в чистом виде
- Особенно в спринге, особенно для большого количества бинов
- Поэтому придумали аспектно-ориентированное программирование (АОР, АОП)
- В отличие от, например, функционального программирования, это расширение, а не новая парадигма

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет



Маршрут вебинара



Cross-cutting

Aspects, Proxy

Weaving

Состав аспекта

Advices

AspectJ pointcuts

Нативные аспекты

Best, Bad и Быт Practices



Понятие Аспекта и прокси

Аспект

- Аспект – это специальный класс, который описывает, какую cross-cutting логику применить в заданных (в нем же) точках вашего приложения
- Аспекты пишутся на разных технологиях
- Например, AspectJ
- Spring AOP – это базовая функциональность Spring для АОП (использующая AspectJ, кстати, но не только)
- На Spring AOP написана половина фреймворков
- Хотя сами Вы редко будете писать аспекты

Аспект

```
package ru.otus.spring.logging;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* ru.otus.spring.dao.BookDaoImpl.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println(
            "Вызов метода : " + joinPoint.getSignature().getName()
        );
    }
}
```

Диаграмма классов шутки

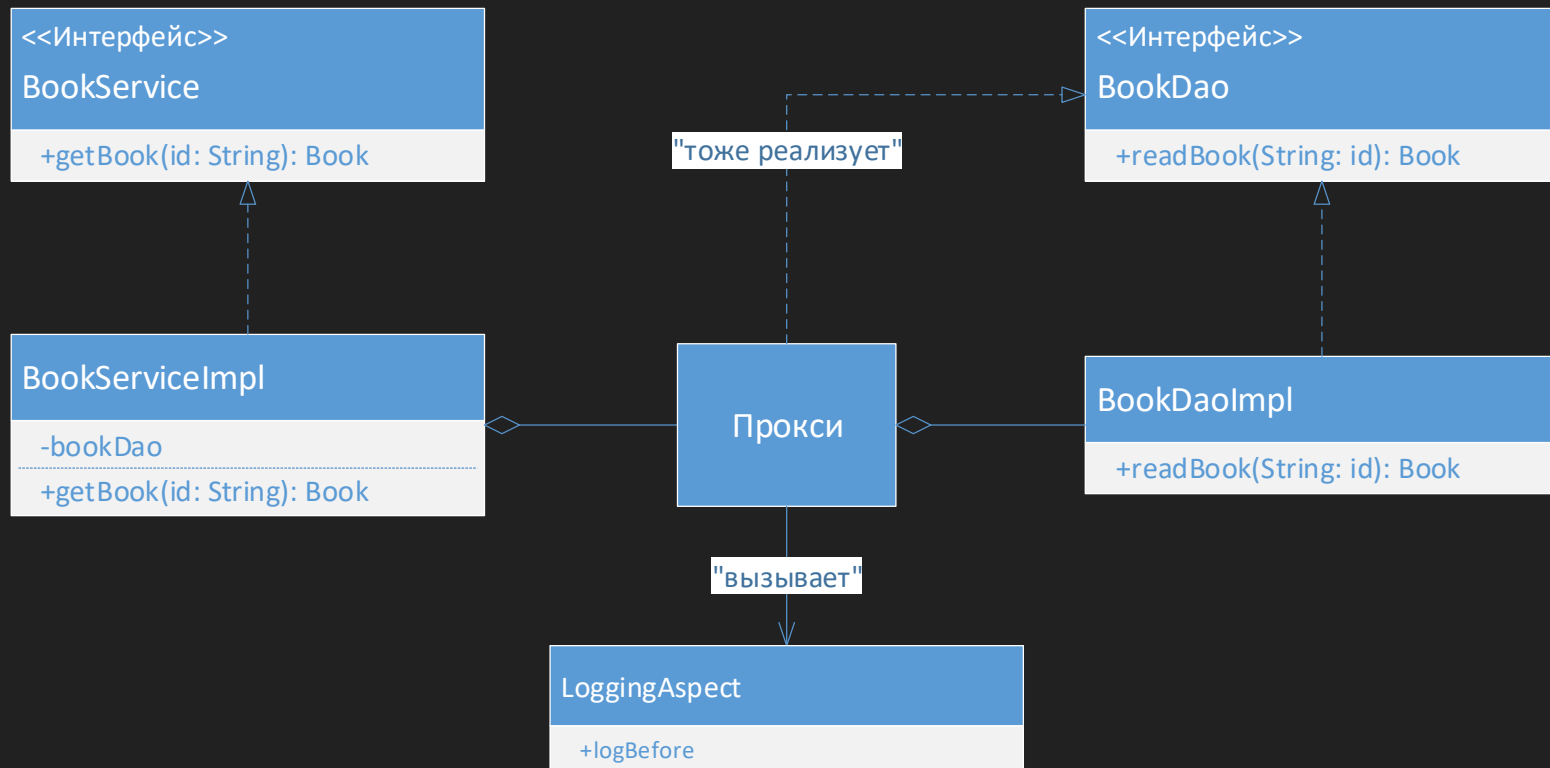
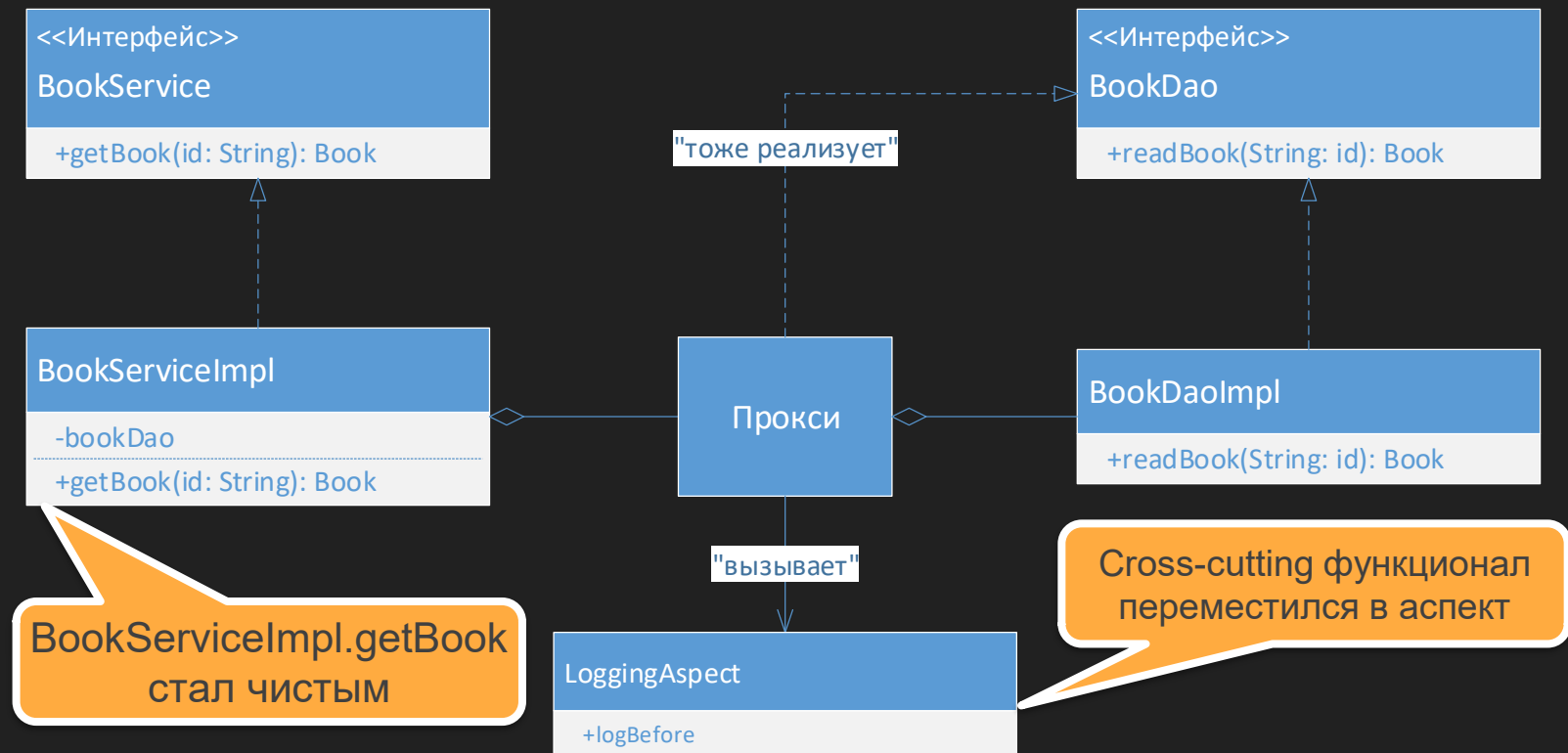


Диаграмма классов шутки



BookServiceImpl теперь

```
@Service
public class BookServiceImpl implements BookService {

    private final BookDao bookDao;

    public BookService(BookDao bookDao) {
        this.bookDao = bookDao;
    }

    @Override
    public Book getBook(String bookId) {
        Book book = bookDao.readBook(bookId);
        return book;
    }
}
```

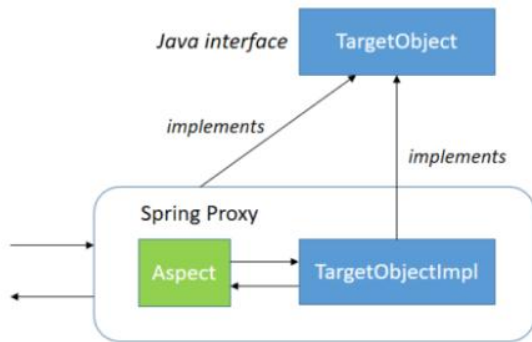
Роемся во внутренностях

- BookDao, BookDaoImpl
- В контексте лежит бин bookDaoImpl
- BookServiceImpl использует интерфейс BookDao
- Spring обманывает бин bookServiceImpl и даёт ему обёртку над bookDaoImpl, которая вызывает все аспекты, а потом вызывает метод bookDaoImpl
- Интерфейс этой обёртки – тоже BookDao
- Такая обёртка называется «прокси» (proxy)
- Обёрток вокруг метода может несколько

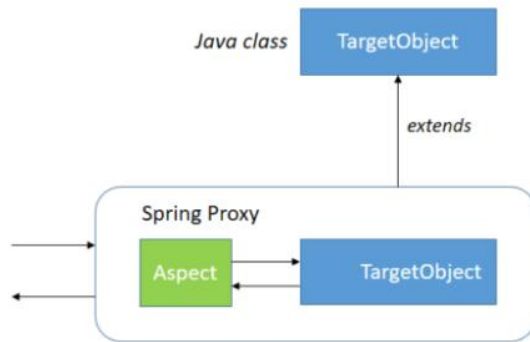
Proxy

Spring AOP Process

JDK Proxy (interface based)



CGLib Proxy (class based)



Аспект

- JDK proxy умеет только публичные
- Cglib умеет оборачивать и protected
- Но Spring AOP даже с Cglib не делает всего, а имитирует JDK proxy
- Таким образом вне зависимости от типа прокси – у Вас одно и то же поведение
- Т.е. можно не париться и знать что только публичные
- Прокси для JPA сущностей работают и с protected. Но это отдельная история 😊

Демо (прокси)

Запустите клиента, посмотрите код

1. `git clone git@github.com:OtusTeam/Spring.git`
`cd ./<имя группы>/spring-04-aop`
2. Запускаем `spring-04-aop/aop-demo/aop-custom/MainSimple`

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет



Маршрут вебинара



Cross-cutting

Aspects, Proxy

Weaving

Состав аспекта

Advices

AspectJ pointcuts

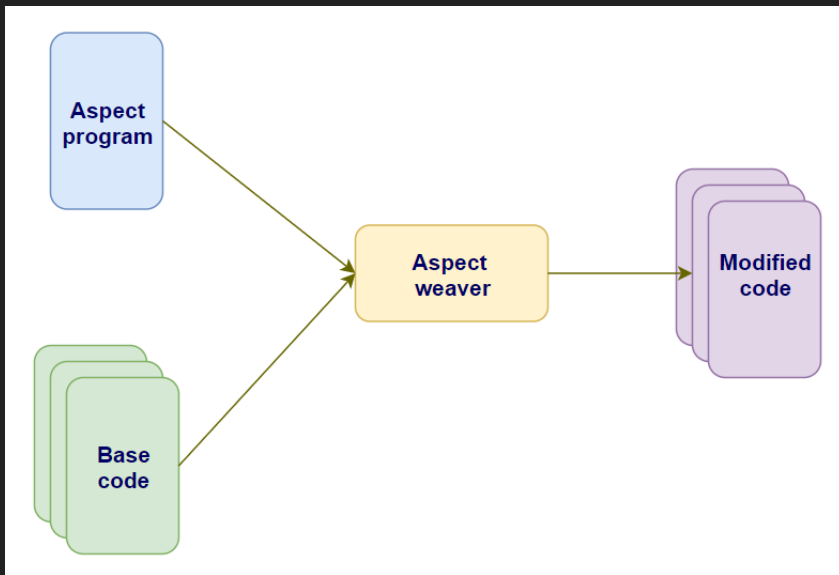
Нативные аспекты

Best, Bad и Быт Practices

Weaving

Weaving

Weaving – это процесс применения (связывания, объединения) кода аспекта к оригинальному коду (JoinPoint)



Виды Weaving-a

- Compile time weaving – аспекты внедряются в классы на этапе компиляции. Делается через плагин в мавен. @Component над аспектами не нужен
- Load time weaving – аспекты внедряются в классы на этапе загрузки в JVM. Делается через инструментацию (агент). Дополнительно нужен aop.xml. @Component над аспектами не нужен. Спринг может участвовать. У него есть свой агент и можно включать/выключать weaving через аннотацию @EnableLoadTimeWeaving
- Runtime weaving – аспекты это бины внутри прокси. @Component над аспектами обязателен. Это как раз то что используют в спринге. За него отвечает зависимость aspectj-weaver

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет

Маршрут вебинара



Cross-cutting

Aspects, Proxy

Weaving

Состав аспекта

Advices

AspectJ pointcuts

Нативные аспекты

Best, Bad и Быт Practices

Состав аспекта

Состав аспекта

- Мы видели, что сквозная логика в аспектах “хранится” в методах
- Такие методы называются Advice (совет). Они бывают разных видов, но об этом позже
- Advice-ы применяются к приложению в т.н. JoinPoint (точка соединения). Это место (точка) в программе, нуждающаяся в дополнительной логике. В большинстве случаев это вызов метода
- Правило/описание/шаблон, по которому можно узнать подходит JoinPoint текущему Advice называется PointCut (срез)

Разберёмся в коде

- Да, раньше в примере были BookDaoImpl, BookServiceImpl
- А теперь PersonService и PersonDaoSimple
- Код можно также посмотреть в репозитории с примерами

Разберёмся в коде

```
package ru.otus.spring.logging;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* ru.otus.spring.dao.PersonDaoSimple.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println(
            "Вызов метода : " + joinPoint.getSignature().getName()
        );
    }
}
```



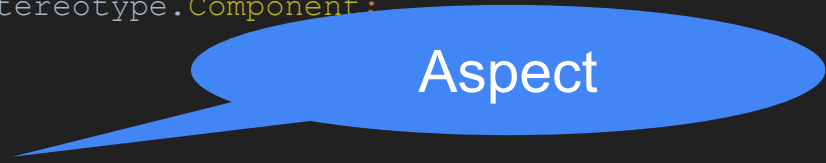
Разберёмся в коде

```
package ru.otus.spring.logging;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* ru.otus.spring.dao.PersonDaoSimple.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println(
            "Вызов метода : " + joinPoint.getSignature().getName()
        );
    }
}
```



Aspect

Разберёмся в коде

```
package ru.otus.spring.logging;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* ru.otus.spring.dao.PersonDaoSimple.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println(
            "Вызов метода : " + joinPoint.getSignature().getName()
        );
    }
}
```



Advice

Разберёмся в коде

```
package ru.otus.spring.service;

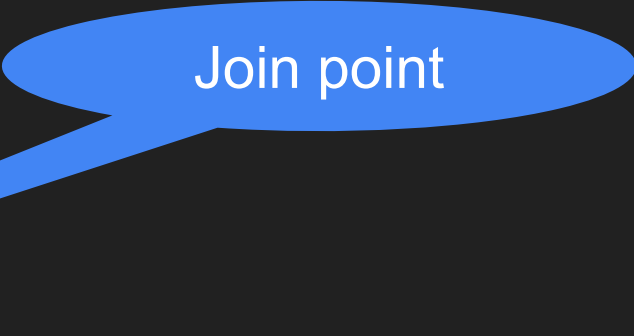
import org.springframework.stereotype.Service;
import ru.otus.spring.dao.PersonDao;
import ru.otus.spring.domain.Person;

@Service
public class PersonServiceImpl implements PersonService {

    private final PersonDao dao;

    public PersonServiceImpl(PersonDao dao) {
        this.dao = dao;
    }

    public Person getByName(String name) {
        return dao.findByName(name);
    }
}
```



Join point

Разберёмся в коде

```
package ru.otus.spring.logging;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* ru.otus.spring.dao.PersonDaoSimple.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println(
            "Вызов метода : " + joinPoint.getSignature().getName()
        );
    }
}
```

Join point

Join point

Разберёмся в коде

```
package ru.otus.spring.logging;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;

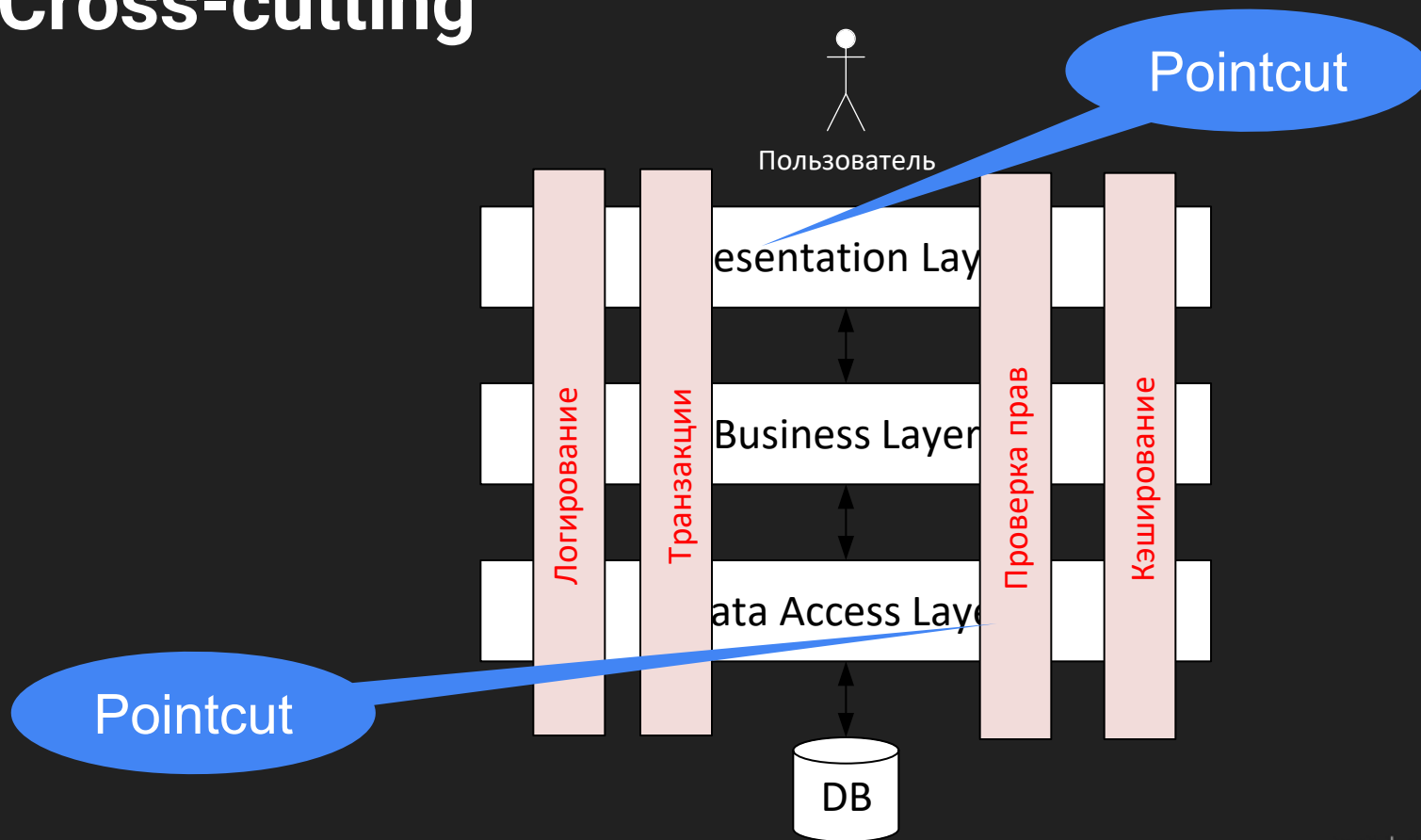
@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* ru.otus.spring.dao.PersonDaoSimple.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println(
            "Вызов метода : " + joinPoint.getSignature().getName()
        );
    }
}
```



Pointcut

Cross-cutting



Чтобы заработало

- Добавить зависимости для aspectj (aspectjrt + aspectjweaver). spring-aop транзитивна spring-context. Ее можно не писать
- Написать аспект
- @Aspect + @Component на аспекте!
- @Order (если нужно), чтобы определить порядок применения
- Аннотации advice-ов с pointcut-ом (@Before и т.д.)
- @EnableAspectJAutoProxy на классе конфигурации (со Spring Boot 2.1.0 не нужно ставить)

Аннотации аспекта

```
package ru.otus.spring.logging;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

@Order(10)
@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* ru.otus.spring.dao.PersonDaoSimple.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println(
            "Вызов метода : " + joinPoint.getSignature().getName()
        );
    }
}
```


@EnableAspectJAutoProxy

```
package ru.otus.spring;

import org.springframework.context.annotation.*;
import ru.otus.spring.domain.Person;
import ru.otus.spring.service.PersonService;

@EnableAspectJAutoProxy
@Configuration
@ComponentScan
public class Main {

    public static void main(String[] args) {
        // ...
    }
}
```

Упражнение 1 (порядок аспектов)

В каком @Order нужно «оборачивать» следующие аспекты:

- Логгирование
- Авторизация
- Exception Handling
- Транзакции
- Кэширование
- Benchmarking

Немного про @Order

```
int LOWEST_PRECEDENCE = Integer.MAX_VALUE;
```

```
public @interface Order {  
    int value() default Ordered.LOWEST_PRECEDENCE;  
}
```

```
// Чем выше value в Order,  
// тем ближе аспект к оригинальному методу  
// тем позже он будет выполнен
```

```
Аспект с @Order(1) {  
    Аспект с @Order(2) {  
        Оригинальный метод  
    }  
}
```

Итоги темы AOP

- Cross-cutting – вспомогательная пересекающая функциональность
- AOP – расширение ООП, только для ортогональной функциональности
- Можно писать на разных технологиях, есть Spring AOP, использующий AspectJ



Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет



Маршрут вебинара



Cross-cutting

Aspects, Proxy

Weaving

Состав аспекта

Advices

AspectJ pointcuts

Нативные аспекты

Best, Bad и Быт Practices

Advices

План темы Advice

- Advice
- Виды Advice-ов
- Их синтаксис

Advice

- Advice – это те действия, которые выполняются помимо Jointpoint
- В аспекте может быть несколько Advice-ов
- Напомню, что на метод может быть навешено несколько Advice-ов и, возможно из разных аспектов
- Аспекты можно упорядочивать, advice-ы не всегда
- Advice бывают разные

Виды @Advice-ов

- @Before – выполняется перед точкой входа (есть возможность и не заходить в метод)
- @After – выполняется после точки входа (даже после исключения)
- @Around – до и после (вокруг)
- @AfterReturning – после того, как точка входа завершилась корректно
- @AfterThrowing – в случае исключения в точке входа

@Before advice

```
package ru.otus.spring.logging;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* ru.otus.spring.dao.PersonDaoSimple.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println(
            "Логгирование перед вызовом метода : " +
            joinPoint.getSignature().getName()
        );
    }
}
```

@After advice

```
package ru.otus.spring.logging;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.After;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @After("execution(* ru.otus.spring.dao.PersonDaoSimple.*(..))")
    public void logAfter(JoinPoint joinPoint) {
        System.out.println(
            "Логгирование после вызова метода : " +
            joinPoint.getSignature().getName()
        );
    }
}
```

@AfterReturning advice

```
package ru.otus.spring.logging;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.AfterReturning;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @AfterReturning(
        pointcut = "execution(* ru.otus.spring.dao.PersonDaoSimple.*(..))",
        returning = "result"
    )
    public void logAfterReturning(JoinPoint joinPoint, Object result) {
        System.out.println(
            "Логгирование после возврата значения из: " +
            joinPoint.getSignature().getName()
        );
        System.out.println("Метод вернул: " + result);
    }
}
```

@AfterThrowing advice

```
package ru.otus.spring.logging;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.AfterThrowing;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @AfterThrowing(
        pointcut = "execution(* ru.otus.spring.dao.PersonDaoSimple.*(..))",
        throwing = "error"
    )
    public void logAfterThrowing(JoinPoint joinPoint, Throwable error) {
        System.out.println(
            "Логгирование выброса исключения значения методом: " +
            joinPoint.getSignature().getName()
        );
        System.out.println("Возникшее исключение: " + error);
    }
}
```

@Around advice

```
package ru.otus.spring.logging;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Around;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Around("execution(* ru.otus.spring.dao.PersonDaoSimple.*(..))")
    public Object logAround(ProceedingJoinPoint joinPoint) throws Throwable {
        System.out.println("Логгирование перед вызовом метода:");
        System.out.println(joinPoint.getSignature().getName());
        System.out.println("Аргументы метода:");
        System.out.println(Arrays.toString(joinPoint.getArgs()));

        var result = joinPoint.proceed();

        System.out.println("Логгирование после вызова метода");
        return result;
    }
}
```

Advice-ы

- Вы можете глушить Exception-ы
- Вы можете генерировать Exceptions
- Вы можете вызывать метод по условию
- Вы можете изменять аргументы и возвращаемые значения
- Половина Spring-а написана на Advice

Упражнение 2 (виды advice-ов)

С помощью каких advice-ов можно добавить следующую логику?

- Логгирование
- Авторизация
- Exception Handling
- Транзакции
- Кэширование
- Benchmarking

Итоги темы Advice

- Advice – это дополнительный код, который будет выполняться помимо JoinPoint (или вместо)
- Advice бывает нескольких видов: @Before, @After, @Around, @AfterReturning, @AfterThrowing
- И теперь нам знаком их синтаксис

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет

Маршрут вебинара



Cross-cutting

Aspects, Proxy

Weaving

Состав аспекта

Advices

AspectJ pointcuts

Нативные аспекты

Best, Bad и Быт Practices

AspectJ pointcuts



План темы Pointcuts

- Pointcuts
- AspectJ Pointcuts

Pointcuts

- Pointcuts – язык описания мест Join point
- В Spring AOP принят синтаксис AspectJ
- С pointcut-ами можно вытворять многое

Немного упражнений

- Предлагаю посмотреть на экран и попробовать поугадывать.
- Слушали, кто смотрит нас в записи – могут гуглить.

Упражнение 3 (смотрим pointcuts)

// метод прям конкретный публичный

```
"execution(public String ru.otus.dao.*.findById(Long))"
```

// все публичные методы в классе FooDao

```
"execution(* ru.otus.dao.FooDao.*(..))"
```

// все публичные методы во всех классах пакета ru.otus.dao

```
"execution(* ru.otus.dao.*.*(..))"
```

// публичные методы в классе

```
"execution(* ru.otus.dao.FooDaoImpl.*(..))"
```

// то же самое

```
"within(ru.otus.dao.FooDaoImpl)"
```

// все методы внутри классов внутри пакетей

```
"within(ru.otus.dao..*)"
```

// методы в классах реализующие интерфейс

```
"target(ru.otus.MyInterface)"
```

// методы в классах. Помеченных аннотацией

```
"@target(org.springframework.stereotype.Repository)"
```

// методы, помеченные аннотацией

```
"@annotation(org.baeldung.aop.annotations.Loggable)"
```

Язык pointcuts

- `execution` – применяется ко всем публичным методам, которые совпадают по сигнатуре
- `within` – применяется ко всем публичным методам у объектов заданного типа или у классов, имена которых удовлетворяют заданному паттерну
- `target` – применяется ко всем публичным методам классов, которые являются экземпляром заданного класса/интерфейса. Шаблон передавать нельзя
- `this` – применяется ко всем публичным методам классов, у которых прокси заданного типа
- `bean` – применяется ко всем публичным методам классов, у которых `id` бина соответствует заданному паттерну
- `args` – применяется ко всем публичным методам, которые совпадают по сигнатуре аргументов. Аргументы определены своими типами

Язык pointcuts

- `@target` – применяется ко всем публичным методам классов, которые имеют над собой заданную аннотацию
- `@within` – применяется ко всем публичным методам классов, которые имеют над собой заданную аннотацию
- `@annotation` – применяется ко всем публичным методам, которые имеют над собой заданную аннотацию
- `@args` – применяется ко всем публичным методам, которые совпадают по сигнатуре аргументов. Аргументы определены своими аннотациями

`@within` и `@target` в Spring AOP по сути делают одно и то же, но в чистом AspectJ `@within` умеет в статические методы:

<https://stackoverflow.com/questions/51124771/difference-between-target-and-within-spring-aop>

Подробнее про язык pointcut-ов

- <http://www.baeldung.com/spring-aop-pointcut-tutorial>
- <https://docs.spring.io/spring-framework/docs/current/spring-framework-reference/core.html#aop-pointcuts-designators>
- <https://howtodoinjava.com/spring-aop/aspectj-pointcut-expressions/>

Pointcuts на аннотациях

```
"@target(org.springframework.stereotype.Repository) "
```

```
"@annotation(org.baeldung.aop.annotations.Loggable) "
```

- Являются предпочтительными
- @Transactional, @Repository, @Controller так и работают
- Правила приличия – они делают наличие аспекта явным

Упражнение 4 (аспект логгирования)

Залоггировать факт вызова методов `PersonDaoSimple`.
(сам класс `PersonDaoSimple` трогать не нужно)

1. `git clone git@github.com:OtusTeam/Spring.git`
2. `cd ./<имя группы>/spring-04-aop/aop-classwork`
3. `@EnableAspectJAutoProxy, @Aspect, @Component, @Before`
4. Число 4 в чат, если сделаете

Слушатели, которые смотрят нас в записи, могут поставить паузу

@Before advice

```
package ru.otus.spring.logging;

import org.aspectj.lang.JoinPoint;
import org.aspectj.lang.annotation.Aspect;
import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* ru.otus.spring.dao.PersonDaoSimple.*(..))")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println(
            "Логгирование перед вызовом метода : " +
            joinPoint.getSignature().getName()
        );
    }
}
```

Конец упражнения

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет

Итоги темы Pointcuts

- Pointcuts – описание мест, куда применить Advice
- В Spring принят синтаксис AspectJ Pointcuts
- Но внутри используется нативный AOP

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет



Маршрут вебинара

Cross-cutting

Aspects, Proxy

Weaving

Состав аспекта

Advices

AspectJ pointcuts

Нативные аспекты

Best, Bad и Быт Practices



Native Spring AOP

Native Spring AOP

- Внутри Spring использует свой способ создания аспектов
- В нем Pointcut это интерфейс, реализация которого определяет какой класс или метод подходит для применения advice
- Advice – это тоже интерфейс. Реализация будет содержать нашу cross-cutting логику
- PointcutAdvisor – служит для объединения Pointcut + Advice (getAdvice()/getPointCut())
- DefaultAdvisorAutoProxyCreator – создает из бинов PointcutAdvisor-ов прокси. По сути weaving
- Бины Spring-a создает InfrastructureAdvisorAutoProxyCreator

Подробнее тут: <https://docs.spring.io/spring-framework/docs/current/spring-framework-reference/core.html#aop-api>

Демо

Маршрут вебинара

Cross-cutting

Aspects, Proxy

Weaving

Состав аспекта

Advices

AspectJ pointcuts

Нативные аспекты

Best, Bad и Быт Practices



Best, Bad и БЫТ Practices

Как применяется аспекты в Spring

- Только для методов бинов, которые лежат к контексте!
- Только для публичных методов! На private – не заработает!
- Только если Вы вызываете из одного бина другой бин!
- Cglib-проху предоставляет такие возможности, но Spring сделал так, чтобы не было разницы между тем, как проху созданы
- У спринга есть и другие способы создания прокси 😊

Жизнь тяжела

- Аспекты тяжело воспринимаются
- Тяжело пишутся хорошие
- Ещё тяжелее дебажатся
- А тем более покрываются тестами
- Ошибки в аспектах почти невозможно найти

Best-Practices

- Делайте аспекты как можно более прозрачными
- `@Annotation pointcut`-ы отличный способ
- Максимально отлаживаются до того как появится в коде
- Используя спринговую аннотацию помните, что чаще всего там аспект
- Аспект, как паттерн, – средство решения проблемы, а не её создания 😊

Bad-Practices

- Не делайте много неявного в аспекте
- Не навешивайте на абсолютно все методы
- Аспект по идее должен быть быстрый
- Ни в коем случае не пишите бизнес-логику на аспектах, они только для cross-cutting

Быт Practices

Примеры использования аспектов:

- Вы хотите залоггировать, какой пользователь вызывает @Deprecated методы
- Вы не хотите пускать сервисы из чёрного списка к методам
- Аудит лог – что изменилось, какие сервисы в итоге вызвались пользователем
- Настройки дополнительной обработки (валидации)



Итоги

Итоги вебинара

1.	Разбирались, что такое аспекты
2.	Разобрались, как работают аспекты в Spring (это ключевая функциональность)
3.	Разобрались, как писать на аспектах cross-cutting функциональность
4.	А бизнес-логику мы не будем писать на аспектах 😊

Домашнее задание

- НЕТ его 😊
- Доделывайте спокойно приложение тестирования
- А можете написать аспект логгирования
- Его можно сделать в текущем, еще не сданном ДЗ
- Но если хотите, чтобы он был проверен, лучше дополнительно указать это в чате задания

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет



**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**

<https://otus.ru/polls/146974/>

Спасибо за внимание!

Аспектов Вам!



Владимир Иванов

Senior Java Developer

Telegram: @ivanovvl

